

Лабораторна робота № 1

Максимізація функції генетичним алгоритмом

Виконав: Самусь Дем'ян Михайлович

Група: ММШІ-2

Репозиторій: github.com/Kentoso/gen-algo-labs/lab1

Мета та постановка задачі

Реалізувати генетичний алгоритм для пошуку найбільшого значення функції:

$$f(w, x, y, z) = w^3 + x^2 - y^2 - z^2 + 2yz - 3wx + wz - xy + 2.$$

Кожна зі змінних є цілим числом від 0 до 15, тобто на неї вистачає рівно чотирьох бітів. Через це хромосома має довжину 16 бітів і складається з чотирьох послідовних блоків, які відповідають w , x , y та z . У програмі її збережено як звичайний список нулів і одиниць, причому старший біт кожного блоку стоїть зліва. Наприклад, запис 1101 | 0110 | 0111 | 1100 читається як $w = 13$, $x = 6$, $y = 7$, $z = 12$.

Алгоритм та обрані параметри

Стартова популяція складається зі 100 випадкових хромосом. За fitness взято безпосередньо значення заданої функції, тому чим воно більше, тим кращим вважається розв'язок. Від'ємні значення тут нічим не заважають і нормалізувати їх не довелося: турнірний відбір лише порівнює особини між собою і не будує ймовірностей, пропорційних до fitness.

Параметр	Значення
Розмір популяції	100
Максимальна кількість поколінь	500
Частка учасників турніру	5% популяції, округлення вгору
Crossover	Рівномірний, імовірність 1.0
Mutation	Імовірність 0.05 на кожного нащадка; інверсія одного біта
Елітизм	1 найкраща особина
Межа стагнації	5 поколінь без строгого покращення
Seed	43
Період виведення статистики	Кожне покоління

Щоб вибрати кожного з двох батьків, програма випадково бере 5 різних позицій популяції, і перемагає та особина, у якої fitness найбільший. Турніри між собою незалежні, тому одна й та сама особина може стати обома батьками. Однакові

хромосоми в популяції теж дозволені. Чим більший турнір, тим сильніша перевага кращих особин, а разом з нею і ризик швидше втратити різноманіття.

Рівномірний crossover працює так: для кожної позиції незалежно, з імовірністю 0.5, біти двох батьків міняються місцями між двома нащадками. Імовірність crossover дорівнює 1.0, тобто оператор застосовується до кожної пари без винятку. Далі кожен нащадок з імовірністю 0.05 мутує, і тоді інвертується рівно один випадковий біт. Ця ймовірність стосується всієї особини, а не кожного біта окремо.

Кожне наступне покоління починається з незміненої копії найкращої особини, а решту місць займають нащадки. Якщо вільним лишається тільки одне місце, береться перший нащадок пари, тому розмір популяції залишається сталим. Елітна особина не проходить ні через crossover, ні через mutation, тому найкращий fitness ніколи не падає, а середній може коливатися.

Після кожного покоління програма дивиться, чи покращився найкращий fitness. Лічильник стагнації обнуляється лише за строгого покращення, а однакове значення його не скидає. Робота завершується або після 5 поколінь без покращення, або після 500 поколінь. Початкову популяцію позначено як покоління 0.

Запуск і результат

У каталозі `lab1` виконати:

```
uv run main.py
```

Усі параметри винесено в іменовані константи на початку `main.py`, тож змінювати їх зручно в одному місці. Жодних залежностей поза стандартною бібліотекою немає. Наведені нижче результати перевірено запуском `python3 main.py`.

На кожній контрольній точці програма друкує найкращий і середній fitness, а також п'ять найкращих особин поточного покоління разом з їхнім fitness, хромосомами та декодованими змінними. Особини впорядковані за спаданням fitness. Однакові хромосоми навмисно не прибрано з виведення, бо саме вони показують, як популяція сходиться. Якщо особин менше п'яти, виводяться всі наявні. За поточних налаштувань статистика друкується щопокоління, і завдяки цьому добре видно ранні покращення. Для описаних параметрів отримано таке:

```
Generation 0 | Best: 3475 | Average: 738.00
#1 | Fitness: 3475 | 1111 | 0001 | 0111 | 1011 | w = 15, x = 1, y = 7, z = 11
#2 | Fitness: 3229 | 1111 | 0111 | 1000 | 1110 | w = 15, x = 7, y = 8, z = 14
#3 | Fitness: 3005 | 1111 | 1011 | 1110 | 1011 | w = 15, x = 11, y = 14, z = 11
#4 | Fitness: 2953 | 1111 | 1111 | 0011 | 0101 | w = 15, x = 15, y = 3, z = 5
#5 | Fitness: 2845 | 1110 | 0001 | 0111 | 1110 | w = 14, x = 1, y = 7, z = 14
Generation 1 | Best: 3475 | Average: 2037.77
#1 | Fitness: 3475 | 1111 | 0001 | 0111 | 1011 | w = 15, x = 1, y = 7, z = 11
#2 | Fitness: 3415 | 1111 | 0001 | 0010 | 1011 | w = 15, x = 1, y = 2, z = 11
#3 | Fitness: 3311 | 1111 | 0101 | 1000 | 1110 | w = 15, x = 5, y = 8, z = 14
```

```

#4 | Fitness: 3291 | 1111 | 0010 | 0000 | 0000 | w = 15, x = 2, y = 0, z = 0
#5 | Fitness: 3261 | 1111 | 0101 | 0011 | 1001 | w = 15, x = 5, y = 3, z = 9
Generation 2 | Best: 3526 | Average: 2984.69
#1 | Fitness: 3526 | 1111 | 0000 | 0111 | 1011 | w = 15, x = 0, y = 7, z = 11
#2 | Fitness: 3475 | 1111 | 0001 | 0111 | 1011 | w = 15, x = 1, y = 7, z = 11
#3 | Fitness: 3471 | 1111 | 0010 | 1010 | 1111 | w = 15, x = 2, y = 10, z = 15
#4 | Fitness: 3458 | 1111 | 0000 | 0011 | 0110 | w = 15, x = 0, y = 3, z = 6
#5 | Fitness: 3433 | 1111 | 0000 | 0000 | 1000 | w = 15, x = 0, y = 0, z = 8
Generation 3 | Best: 3602 | Average: 3247.82
#1 | Fitness: 3602 | 1111 | 0000 | 1111 | 1111 | w = 15, x = 0, y = 15, z = 15
#2 | Fitness: 3538 | 1111 | 0000 | 0111 | 1111 | w = 15, x = 0, y = 7, z = 15
#3 | Fitness: 3526 | 1111 | 0000 | 0111 | 1011 | w = 15, x = 0, y = 7, z = 11
#4 | Fitness: 3526 | 1111 | 0000 | 0111 | 1011 | w = 15, x = 0, y = 7, z = 11
#5 | Fitness: 3513 | 1111 | 0001 | 1001 | 1111 | w = 15, x = 1, y = 9, z = 15
Generation 4 | Best: 3602 | Average: 3380.85
#1 | Fitness: 3602 | 1111 | 0000 | 1111 | 1111 | w = 15, x = 0, y = 15, z = 15
#2 | Fitness: 3586 | 1111 | 0000 | 1111 | 1110 | w = 15, x = 0, y = 15, z = 14
#3 | Fitness: 3586 | 1111 | 0000 | 1011 | 1111 | w = 15, x = 0, y = 11, z = 15
#4 | Fitness: 3571 | 1111 | 0000 | 1010 | 1110 | w = 15, x = 0, y = 10, z = 14
#5 | Fitness: 3571 | 1111 | 0000 | 1010 | 1110 | w = 15, x = 0, y = 10, z = 14
Generation 5 | Best: 3602 | Average: 3491.71
#1 | Fitness: 3602 | 1111 | 0000 | 1111 | 1111 | w = 15, x = 0, y = 15, z = 15
#2 | Fitness: 3602 | 1111 | 0000 | 1111 | 1111 | w = 15, x = 0, y = 15, z = 15
#3 | Fitness: 3602 | 1111 | 0000 | 1111 | 1111 | w = 15, x = 0, y = 15, z = 15
#4 | Fitness: 3602 | 1111 | 0000 | 1111 | 1111 | w = 15, x = 0, y = 15, z = 15
#5 | Fitness: 3602 | 1111 | 0000 | 1111 | 1111 | w = 15, x = 0, y = 15, z = 15
Generation 6 | Best: 3602 | Average: 3564.03
#1 | Fitness: 3602 | 1111 | 0000 | 1111 | 1111 | w = 15, x = 0, y = 15, z = 15
#2 | Fitness: 3602 | 1111 | 0000 | 1111 | 1111 | w = 15, x = 0, y = 15, z = 15
#3 | Fitness: 3602 | 1111 | 0000 | 1111 | 1111 | w = 15, x = 0, y = 15, z = 15
#4 | Fitness: 3602 | 1111 | 0000 | 1111 | 1111 | w = 15, x = 0, y = 15, z = 15
#5 | Fitness: 3602 | 1111 | 0000 | 1111 | 1111 | w = 15, x = 0, y = 15, z = 15
Generation 7 | Best: 3602 | Average: 3562.67
#1 | Fitness: 3602 | 1111 | 0000 | 1111 | 1111 | w = 15, x = 0, y = 15, z = 15
#2 | Fitness: 3602 | 1111 | 0000 | 1111 | 1111 | w = 15, x = 0, y = 15, z = 15
#3 | Fitness: 3602 | 1111 | 0000 | 1111 | 1111 | w = 15, x = 0, y = 15, z = 15
#4 | Fitness: 3602 | 1111 | 0000 | 1111 | 1111 | w = 15, x = 0, y = 15, z = 15
#5 | Fitness: 3602 | 1111 | 0000 | 1111 | 1111 | w = 15, x = 0, y = 15, z = 15
Generation 8 | Best: 3602 | Average: 3579.86
#1 | Fitness: 3602 | 1111 | 0000 | 1111 | 1111 | w = 15, x = 0, y = 15, z = 15
#2 | Fitness: 3602 | 1111 | 0000 | 1111 | 1111 | w = 15, x = 0, y = 15, z = 15
#3 | Fitness: 3602 | 1111 | 0000 | 1111 | 1111 | w = 15, x = 0, y = 15, z = 15
#4 | Fitness: 3602 | 1111 | 0000 | 1111 | 1111 | w = 15, x = 0, y = 15, z = 15
#5 | Fitness: 3602 | 1111 | 0000 | 1111 | 1111 | w = 15, x = 0, y = 15, z = 15

Stopped after 8 generations: stagnation limit reached.
Best chromosome: 1111 | 0000 | 1111 | 1111
w = 15, x = 0, y = 15, z = 15
Best fitness: 3602
First found at generation: 3

```

Якщо підставити знайдені значення в функцію вручну, дістанемо $3375 - 225 - 225 + 450 + 225 + 2 = 3602$, тобто результат збігається. Завдяки фіксованому seed той самий запуск можна повторити в тому ж середовищі і отримати ідентичний вивід.

Висновок

У ході виконання роботи реалізовано пошук максимуму функції $f(w, x, y, z)$ генетичним алгоритмом на двійковому кодуванні, де кожна змінна займає чотири біти хромосоми. Знайдено найбільше значення 3602 при $w = 15$, $x = 0$, $y = 15$, $z = 15$, і його перевірено прямою підстановкою в функцію.

Розібрано та запрограмовано всі стадії генетичного алгоритму: кодування й декодування розв'язку, обчислення fitness, турнірний відбір батьків, рівномірний crossover, мутацію інверсією біта, елітизм і критерій зупинки за стагнацією. Роботу алгоритму продемонстровано на покроковому виведенні статистики: видно, як найкращий fitness зростає з 3475 до 3526 і далі до 3602, а середній підтягується до найкращого в міру збіжності популяції.

Генетичний алгоритм є евристикою, тому зупинка за стагнацією не доводить, що знайдено саме глобальний максимум, а результат залежить від початкової популяції, випадкових операцій і обраних параметрів.

Додаток: повний код `main.py`

```
import math
import random

POPULATION_SIZE = 100
MAX_GENERATIONS = 500
TOURNAMENT_FRACTION = 0.05
CROSSOVER_PROBABILITY = 1.0
MUTATION_PROBABILITY = 0.05
ELITE_COUNT = 1
STAGNATION_LIMIT = 5
RANDOM_SEED = 43
LOG_INTERVAL = 1

BITS_PER_VARIABLE = 4
CHROMOSOME_LENGTH = 16

def decode(chromosome: list[int]) -> tuple[int, int, int, int]:
    values = []
    for start in range(0, CHROMOSOME_LENGTH, BITS_PER_VARIABLE):
        value = 0
        for bit in chromosome[start : start + BITS_PER_VARIABLE]:
            value = value * 2 + bit
        values.append(value)
    w, x, y, z = values
    return w, x, y, z

def format_chromosome(chromosome: list[int]) -> str:
    return " | ".join(
        "".join(map(str, chromosome[start : start + BITS_PER_VARIABLE]))
        for start in range(0, CHROMOSOME_LENGTH, BITS_PER_VARIABLE)
    )

def fitness(chromosome: list[int]) -> int:
    w, x, y, z = decode(chromosome)
    return w**3 + x**2 - y**2 - z**2 + 2*y*z - 3*w*x + w*z - x*y + 2

def select_parent(
    population: list[list[int]], rng: random.Random
) -> list[int]:
    size = math.ceil(len(population) * TOURNAMENT_FRACTION)
    contestants = rng.sample(population, size)
    return max(contestants, key=fitness)

def crossover(
    parent1: list[int], parent2: list[int], rng: random.Random
) -> tuple[list[int], list[int]]:
    child1, child2 = parent1.copy(), parent2.copy()
    if rng.random() < Crossover_PROBABILITY:
        for index in range(CHROMOSOME_LENGTH):
            if rng.random() < 0.5:
                child1[index], child2[index] = child2[index], child1[index]
    return child1, child2

def mutate(child: list[int], rng: random.Random) -> None:
    if rng.random() < MUTATION_PROBABILITY:
```

```

        index = rng.randrange(CHROMOSOME_LENGTH)
        child[index] = 1 - child[index]

def print_statistics(generation: int, population: list[list[int]]) -> None:
    scores = [fitness(individual) for individual in population]
    print(
        f"Generation {generation:4d} | Best: {max(scores):5d}"
        f" | Average: {sum(scores) / len(scores):9.2f}"
    )
    top_five = sorted(population, key=fitness, reverse=True)[:5]
    for rank, individual in enumerate(top_five, start=1):
        w, x, y, z = decode(individual)
        print(
            f" #{rank} | Fitness: {fitness(individual):5d}"
            f" | {format_chromosome(individual)}"
            f" | w = {w}, x = {x}, y = {y}, z = {z}"
        )

def validate_settings() -> None:
    for name, value in (
        ("POPULATION_SIZE", POPULATION_SIZE),
        ("MAX_GENERATIONS", MAX_GENERATIONS),
        ("ELITE_COUNT", ELITE_COUNT),
        ("STAGNATION_LIMIT", STAGNATION_LIMIT),
        ("LOG_INTERVAL", LOG_INTERVAL),
    ):
        if type(value) is not int or value < 1:
            raise ValueError(f"{name} must be a positive integer")
    if POPULATION_SIZE < 2:
        raise ValueError("POPULATION_SIZE must be at least 2")
    if ELITE_COUNT >= POPULATION_SIZE:
        raise ValueError("ELITE_COUNT must be smaller than POPULATION_SIZE")
    if not 0 < TOURNAMENT_FRACTION <= 1:
        raise ValueError("TOURNAMENT_FRACTION must be in (0, 1]")
    for name, value in (
        ("CROSSOVER_PROBABILITY", CROSSOVER_PROBABILITY),
        ("MUTATION_PROBABILITY", MUTATION_PROBABILITY),
    ):
        if not 0 <= value <= 1:
            raise ValueError(f"{name} must be in [0, 1]")

def main() -> None:
    validate_settings()
    rng = random.Random(RANDOM_SEED)
    population = [
        [rng.randrange(2) for _ in range(CHROMOSOME_LENGTH)]
        for _ in range(POPULATION_SIZE)
    ]
    best = max(population, key=fitness).copy()
    best_generation = 0
    stagnant_generations = 0
    stop_reason = "generation limit reached"

    print("Lab 1: Function maximization with a genetic algorithm")
    print(f"Population: {POPULATION_SIZE} | Generation limit: {MAX_GENERATIONS}")
    print(
        f"Tournament: {TOURNAMENT_FRACTION:.0%}"
        f" ({math.ceil(POPULATION_SIZE * TOURNAMENT_FRACTION)} contestants)"
        f" | Elite: {ELITE_COUNT}"
    )
    print(
        f"Uniform crossover: {CROSSOVER_PROBABILITY:.0%}"
        f" | Single-bit mutation per child: {MUTATION_PROBABILITY:.0%}"
    )
    print(f"Stagnation limit: {STAGNATION_LIMIT} | Seed: {RANDOM_SEED}")
    print_statistics(0, population)

```

```

generation = 0
for generation in range(1, MAX_GENERATIONS + 1):
    new_population = [best.copy() for _ in range(ELITE_COUNT)]
    while len(new_population) < POPULATION_SIZE:
        parent1 = select_parent(population, rng)
        parent2 = select_parent(population, rng)
        children = crossover(parent1, parent2, rng)
        for child in children:
            if len(new_population) == POPULATION_SIZE:
                break
            mutate(child, rng)
            new_population.append(child)
    population = new_population

    generation_best = max(population, key=fitness)
    if fitness(generation_best) > fitness(best):
        best = generation_best.copy()
        best_generation = generation
        stagnant_generations = 0
    else:
        stagnant_generations += 1

    if generation % LOG_INTERVAL == 0:
        print_statistics(generation, population)
    if stagnant_generations >= STAGNATION_LIMIT:
        stop_reason = "stagnation limit reached"
        break

    if generation % LOG_INTERVAL != 0:
        print_statistics(generation, population)
    w, x, y, z = decode(best)
    print(f"\nStopped after {generation} generations: {stop_reason}.")
    print(f"Best chromosome: {format_chromosome(best)}")
    print(f"w = {w}, x = {x}, y = {y}, z = {z}")
    print(f"Best fitness: {fitness(best)}")
    print(f"First found at generation: {best_generation}")

if __name__ == "__main__":
    main()

```